Assertion Messages with Large Language Models (LLMs) for Code

Ahmed Aljohani
University of North Texas
Denton, USA
ahmedaljohani@my.unt.edu

Anamul Haque Mollah
University of North Texas
Denton, USA
anamulmollah@my.unt.edu

Hyunsook Do
University of North Texas
Denton, USA
hyunsook.do@unt.edu

Abstract

Assertion messages significantly enhance unit tests by clearly explaining the reasons behind test failures, yet they are frequently omitted by developers and automated test-generation tools. Despite recent advancements, Large Language Models (LLMs) have not been systematically evaluated for their ability to generate informative assertion messages. In this paper, we introduce an evaluation of four state-of-the-art Fill-in-the-Middle (FIM) LLMs— Qwen2.5-Coder-32B, Codestral-22B, CodeLlama-13b-hf, and StarCoder—on a dataset of 216 Java test methods containing developer-written assertion messages. We find that Codestral-22B achieves the highest quality score of 2.76 out of 5 using a human-like evaluation approach, compared to 3.24 for manually written messages. Our ablation study shows that including descriptive test comments further improves Codestral's performance to 2.97, highlighting the critical role of context in generating clear assertion messages. Structural analysis demonstrates that all models frequently replicate developers' preferred linguistic patterns. We discuss the limitations of the selected models and conventional text evaluation metrics in capturing diverse assertion message structures. Our benchmark, evaluation results, and discussions provide an essential foundation for advancing automated, context-aware generation of assertion messages in test code. A replication package is available at https://doi.org/10.5281/zenodo.15293133.

Keywords

Unit Testing, Test Code, Assertion Messages, Large language models (LLMs), Fill-in-the-Middle (FIM)

ACM Reference Format:
Ahmed Aljohani, Anamul Haque Mollah, and Hyunsook Do. 2025. Assertion Messages with Large Language Models (LLMs) for Code. In *Evaluation and Assessment in Software Engineering (EASE '25), June 17–20, 2025, Istanbul, Turkiye.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3756681.3757000

1 Introduction

Unit testing– is a crucial practice in software development that ensures the functionality correctness of individual components of a software system. At the heart of unit tests are assertion statements (e.g, Listing 1 line 4), which verify whether the actual output of a program matches the expected outcome [13]. A well-constructed assertion not only determines the success or failure of a test but also provides valuable feedback to developers through assertion messages—descriptive texts that explain why an assertion fails [21]. These messages (e.g, Listing 1 - `"Password dose not match"`) play a significant role in improving test code understandability, readability, and overall quality.

Listing 1: Test Code Example

```
1  public void testCheckPassword() {
2      String e = "Abc_123!";
3      String a =checkPassword("Abc_123!");
4      assertEquals("Password dose not match", e, a);}
```

Despite their importance, assertion messages are often overlooked in test code. Studies have shown that developers rarely include custom messages in assertion methods [31]. Approximately ≈ 6% of 31K test methods contain at least one assertion. This limitation is also common among automated test-generation techniques, including search-based approaches like EvoSuite [9] and transformer-based methods like AthenaTest [32]. For example, EvoSuite often generates multiple assertions per test method without descriptive messages, making the tests challenging to understand [19]. Similarly, AthenaTest, despite being trained on developer-written tests, frequently generates assertions lacking informative messages [4]. Moreover, existing automated tools for generating assertion statements primarily focus on verifying correctness or producing expected outcomes, but they rarely emphasize generating informative assertion messages [11, 24, 35, 37, 39] (See Section 2 for details). This lack of emphasis can lead to what the software engineering community refers to as a test smell[7]—specifically, *Assertion Roulette* [3]—where multiple assertions without clear failure messages complicate debugging efforts and increase developers' manual inspection workload.

Recently, large language models (LLMs) have been explored for test code enhancement [33]. However, even these advanced models often fail to generate descriptive assertion messages [18, 26] *unless* explicitly guided via detailed prompt engineering techniques [8]. While prior research, such as Deljouyi et al. [8], leveraged LLMs to enhance test code through assertion generation messages, they did not explicitly evaluate how effectively LLMs can produce assertion messages. Peruma et al. [21] highlighted developers' preference for assertion messages that (i) clearly describe test failures (expected versus actual outcomes), (ii) are textual and human-readable, and (iii) accurately reflect the failure's cause.

Given the ability of code LLMs to understand and generate human-readable text in tasks such as code summarization[17, 29], it is worth investigating whether they can produce assertion messages as effective as those written by developers.

We formed our motivation into research questions as follows:

(1) **RQ1 (Performance):** How do LLM-generated assertion messages compare to manually written ones?
We evaluate LLMs against developer-written messages and

EASE '25, Istanbul, Turkiye

ACM ISBN 979-8-4007-1385-9/25/06
https://doi.org/10.1145/3756681.3757000

conduct an ablation study to assess the impact of additional contextual information (e.g., descriptive test comments) on generation quality.

(2) **RQ2 (Structure and Linguistics):** How do the structural and linguistic patterns of LLM-generated assertion messages align with developers-written assertion?
We examine the composition and phrasing of generated messages to determine their consistency with developer conventions and preferences.

To investigate these research questions, we utilize a dataset of Java test methods extracted from 20 Java projects, filtering only those with manually written assertion messages as ground truth. We then prompt LLMs to generate assertion messages using test methods (excluding assertion messages) and compare the generated messages against the original ones. Additionally, we explore whether providing additional context—such as test descriptions—enhances the quality of the generated messages.

This paper makes the following contributions:

(1) Empirical Evaluation – We assess the effectiveness of LLMs in generating assertion messages and compare them with human-written messages.
(2) Impact of Contextual Information – We investigate whether providing additional information improves message quality.
(3) Providing a comprehensive replication dataset to support reproducibility and facilitate further research[1].

The remainder of this paper is organized as follows: Section 2 reviews related work on the automated generation of assertion statements. Section 3 outlines the methodology and experimental design. Section 4 presents the results and key findings derived from our evaluation. Section 5 discusses the broader implications of the findings and provides additional insights. Section 6 outlines the potential limitations and threats to the validity of our study. Finally, Section 7 concludes the paper and highlights directions for future work.

2 Related Work

This section reviews studies on automated assertion generation in test code, focusing on the role of assertion messages, assertion accuracy, and the application of language models.

Zamprogno et al. [37] proposed *AutoAssert*, a tool designed to assist developers in creating assertion statements by capturing runtime values of selected variables during testing. Their evaluation showed that 84.5% of the generated assertions aligned with developer intent. However, *AutoAssert* exhibits clear limitations—it lacks support for generating informative, custom assertion messages and is restricted to specific assertion types.

He et al. [11] explored deep learning techniques for automating assertion generation, emphasizing the significance of providing contextual details about the Method Under Test (MUT). They demonstrated that including additional context, particularly focal methods, significantly improves assertion quality. However, their approach still struggles to consistently generate assertions that are logically coherent and semantically meaningful.

Primbs et al. [24] introduced *AsserT5*, an approach fine-tuning the CodeT5 model specifically for assertion generation. *AsserT5* achieved substantial improvements over existing methods, reaching up to 59.5% accuracy and successfully identifying 33 real software bugs. Their study underscored the critical influence of input formatting and the targeted fine-tuning of pretrained code models, which significantly enhanced assertion accuracy and reliability. Despite these advancements, their primary focus remained on correctness rather than clarity or informativeness of assertion messages.

Watson et al. [35] designed *ATLAS*, a deep learning framework that treats assertion generation as a translation task, learning from paired test and target methods to produce complete assertion statements. Although their model showed promising accuracy in predicting assertions similar to developer-written statements, it notably did not address generating informative assertion messages explicitly, potentially limiting its effectiveness in debugging contexts.

Zhang et al. [39] conducted a comprehensive empirical study assessing the effectiveness of LLMs in assertion generation. Comparing five open-source models against traditional assertion generation techniques, they found that LLMs, particularly CodeT5, consistently achieved higher accuracy. However, their research explicitly focused on generating correct assertion statements rather than informative assertion messages, thus not addressing a crucial aspect that significantly impacts debugging efficiency.

Existing approaches in assertion generation have prioritized correctness and accuracy of assertions, often neglecting the critical role played by informative assertion messages. Frequently, these generated assertions lack descriptive messages. Our work directly addresses this significant gap by evaluating the capabilities of state-of-the-art LLMs, specifically assessing their ability to generate clear, informative assertion messages that facilitate debugging and enhance test code maintainability.

3 Research Method

Figure 1 illustrates our methodology. It begins with *Data Preparation*, where we filter Java test methods containing a single assertion from our dataset. For test methods lacking descriptive comments, we generate test-code comments using GPT-4. In the *Inference* phase, we prompt code models (Qwen2.5-Coder-32B, Codestral-22B, CodeLlama-13b-hf, and StarCoder) to generate assertion messages, both without comments and with the generated test comments. Finally, the *Evaluation* phase assesses the performance and structural characteristics of the generated assertion messages using semantic and linguistic methods.

3.1 Data Source

Dataset: To extract assertion messages, we utilized a publicly available dataset [31][2] consisting of 20 well-engineered Java projects. These projects are known for their high-quality coding standards, clear assertion messages, and have been previously used in multiple studies focused on unit test code analysis and evaluation.

Assertion Message Extraction: Following the approach described by Takebayashi et al. [31], we used the JetBrains Software Development Kit (SDK)[3] to extract test classes from the selected

[1] https://doi.org/10.5281/zenodo.15293133

[2] https://zenodo.org/records/7686565

[3] https://github.com/JetBrains/intellij-community

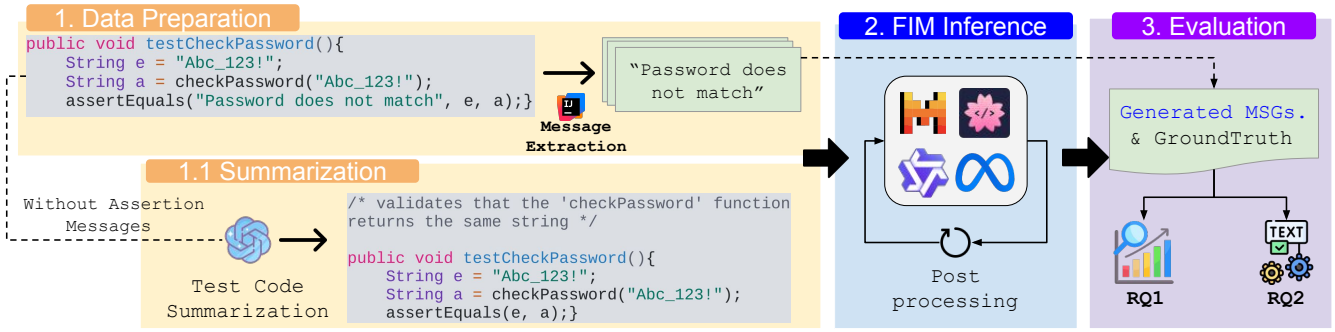

Figure 1: FIM-Based Model Inference Approach.

projects. Specifically, we identified classes containing at least one `@Test` annotation, indicating actual test methods designed to validate the behavior of a Method Under Test (MUT). Subsequently, we parsed these identified classes to extract individual test methods containing exactly one assertion accompanied by a descriptive assertion message. The total number of test methods with a single assertion is 508.

Filtering: To ensure meaningful and reliable ground-truth assertion messages for our evaluation, we applied several preprocessing steps to the extracted dataset. These steps were designed to retain only high-quality assertion messages that clearly provide the reason behind test failures. Specifically, we excluded the following types of assertion messages:

- Empty messages, as they do not provide any information.
- Non-English messages, since our study and the utilized models are primarily focused on English-language assertions.
- Very short or trivial messages, such as single-word assertions or extremely short assertions (less than five characters, e.g., "A"). Such messages have previously been classified as smelly or uninformative assertion messages [31].
- Duplicate messages.

Applying these filtering criteria resulted in a refined dataset consisting of **216** test methods with clear, descriptive, and meaningful assertion messages, forming our final ground-truth set for evaluating model-generated assertion messages. In Table 1, we present the distribution of assertion types in our dataset. The most frequent types are assertEquals, assertTrue, and assertFalse. Although this distribution is derived from individual assertion statements, it closely mirrors the distribution of prior work [31].

Test Code Summarization—Ablation Comparison RQ1: A well-performing model should be able to generate descriptive assertion messages by accurately interpreting the semantic and functional context of the test method. However, providing additional context beyond the raw code—such as comments that describe the test's intent or expected behavior—can significantly enhance the model's understanding and improve the quality of its generated outputs. Prior work by Primbs et al. [24] demonstrated the effectiveness of incorporating method-under-test (MUT) information to improve assertion quality. Similarly, studies in code summarization have shown that enriching source code with code-related information such as data flow and source-code scope leads to better summarization results [1].

In our dataset of 216 test methods, only 65 included developer-written comments. However, many of these were low-quality, such as "TODO" comments [23] or references to external resources (e.g., @link to GitHub links or MUT references). After manual filtering, we retained 43 comments that clearly articulated the test's intent or expected behavior. To maintain consistency and fairness in our evaluation, we generated high-quality descriptive comments for the remaining methods using an LLM approach. We followed the methodology introduced by Sun et al. [28], who showed that GPT-4 (gpt-4-1106-preview) performs well on Java code summarization when prompted with a chain-of-thought (CoT) technique[36], using temperature = 0.1 and top_p = 0.75. We adopted these exact parameters to generate contextually rich comments for methods lacking meaningful documentation.

Table 1: GroundTruth Assertion usage

Assertion Type	Count (%)
assertEquals	80 (37.0%)
assertTrue	68 (31.5%)
assertFalse	31 (14.4%)
assertNotNull	11 (5.1%)
assertThat	11 (5.1%)
assertNull	9 (4.2%)
assertArrayEquals	2 (0.9%)
assertNotSame	2 (0.9%)
assertSame	2 (0.9%)
Total Assertions	**216 (100%)**

3.2 Code Models and Inference

Objective: *Given a complete test method with the assertion message removed, the task is to fill in the missing message within the assertion statement.*

Models: Due to the structured nature of this task—where a specific code segment is missing—we opt to use code models trained with a Fill-in-the-Middle (FIM) objective. FIM-capable models are

particularly well-suited for such tasks, as they are designed to predict code segments situated between a given prefix and suffix [6]. Thus, we evaluate the following base models (i.e., pre-trained models without instruction tuning), selected for their strong performance in recent code generation and infilling benchmarks:

- Qwen2.5-Coder-32B [12][4]
- Codestral-22B-v0.1 [2][5]
- CodeLlama-13B-hf [25][6]
- StarCoder [14][7]

These models have demonstrated strong performance in widely recognized benchmarks such as HumanEvalPlus[8] [16] and the HumanEval Infilling Benchmark[9] by OpenAI.

Listing 2: CodeLlama-13b-hf FIM

```
1 public void testCheckPassword() {
2     String e = "Abc_123!";
3     String a =checkPassword("Abc_123!");
4     //fill_token = tokenizer.fill_token
5     assertEquals("<FILL_ME>", e, a);}
```

Listing 3: Codestral-22B FIM

```
1 prefix = """
2 public void testCheckPassword() {
3      String e = "Abc_123!";
4      String a = checkPassword("Abc_123!");
5      assertEquals("""
6 suffix = """, e, a);}"""
7 //from mistral ... import FIMRequest
8 FIMRequest(prompt=prefix, suffix=suffix)
```

Listing 4: Qwen2.5-Coder-32B FIM

```
1 <|fim_suffix|>
2 public void testCheckPassword() {
3     String e = "Abc_123!";
4     String a =checkPassword("Abc_123!");
5     assertEquals(<|fim_suffix|>, e, a);}
6     <|fim_suffix|>
```

Fill-in-the-Middle (FIM) Inference: To leverage the infilling capabilities of each model, we format the input test methods using the native FIM prompting structure specific to each model. The missing assertion message is replaced with the model's designated placeholder or infill token, enabling it to generate code between a provided prefix and suffix.

For example, in Listing 2 (line 5), we demonstrate FIM usage with CodeLlama-13b-hf, where the assertion message is replaced using the token obtained via `tokenizer.fill_token` (i.e., `<Fill_ME>`). In Listing 3, lines 1 and 6 represent the prefix and suffix segments used with Codestral-22B, while line 8 shows how these components are composed for inference. Listing 4 (lines 1, 5, and 6) illustrates FIM formatting for Qwen2.5-Coder-32B, following the recommended structure provided by its official documentation[10]. StarCoder adopts the same FIM convention as Qwen2.5-Coder-32B, utilizing `<fim-prefix>`, `<fim-suffix>`, and `<fim-middle>` special tokens.

For all models, we used the default hyperparameters provided by their respective HuggingFace configurations during inference. This ensures a fair comparison while preserving the intended behavior of each model.

3.3 Post-processing

Unlike instruct models—where explicit prompt instructions and illustrative examples of the desired output format can make the output reliable—FIM models inherently rely on special native tags that mark positions for content filling. This reliance introduces two key limitations in their generated outputs.

The first limitation is that FIM models occasionally generate extra tokens beyond the intended assertion messages, such as partial or complete code syntax. A common behavior observed during our evaluation is that the models attempt to generate an entire assertion statement rather than simply providing the assertion message. For example, the model might produce an output like:`.."CacheLoader should not be closed", loader.closed);`, combining the assertion message with unintended code fragments. To address this issue, we employ a trimming strategy to isolate and extract only the relevant assertion message portion (e.g., extracting `"CacheLoader should not be closed"`). Most models did not require trimming, except for a few cases observed in CodeLlama and Qwen2.5-Coder.

The second limitation arises when the models fail to generate any content at all for a given test method, resulting in empty outputs. To mitigate this, we apply a retry mechanism: whenever a model initially produces an empty assertion message, we resubmit the prompt to ensure the model attempts to generate the missing content.

Without additional test comments, Qwen2.5-Coder generated 15 empty messages, CodeLlama produced 8, StarCoder produced 3 instances, and Codestral-22B produced none. Incorporating descriptive comments eliminated empty outputs for StarCoder and reduced Qwen's to 13, while CodeLlama remained at 8 and Codestral continued to produce none. These results highlight Codestral's consistent reliability in producing non-empty outputs. Applying one or more retries successfully filled every remaining gap, and increased the reliability of the model-generated assertion messages.

3.4 Evaluation

Performance: Assertion message generation is conceptually similar to code summarization [17], where the input is a code snippet and the output is a concise textual description. To evaluate the performance of LLMs in generating assertion messages, we adopt a combination of text similarity and semantic similarity metrics widely used in code summarization research. Specifically, we use BLEU [20], METEOR [5], and ROUGE-L [15] to measure n-gram overlap between generated and reference messages. To address the limitations of purely lexical metrics [10], we also employ BERTScore [40] to capture deeper semantic similarity. Moreover, recent studies [27, 28] have shown that LLM-generated text can often

[4]https://huggingface.co/Qwen/Qwen2.5-Coder-32B
[5]https://huggingface.co/mistralai/Codestral-22B-v0.1
[6]https://huggingface.co/codellama/CodeLlama-13b-hf
[7]https://huggingface.co/bigcode/starcoder
[8]https://evalplus.github.io/leaderboard.html
[9]https://github.com/openai/human-eval-infilling?tab=readme-ov-file
[10]https://github.com/QwenLM/Qwen2.5-Coder?tab=readme-ov-file

Table 2: Performance of LLMs in Generating Assertion Messages (Average Ground truth LLM-Eval Score = 3.24/5). All n-gram overlap metrics scores are measured using the method described in [29, 38]. ↑ indicates improvement with additional comments. Bold cells highlight highest scores per metric.

	Test Method Only					Test Method with Comments				
Model	BLEU	ROUGE-L	METEOR	BERTScore-F1	LLM-Eval	BLEU	ROUGE-L	METEOR	BERTScore-F1	LLM-Eval
CodeLlama-13b-hf	10.07	25.64	20.84	86.16	2.42	12.26 ↑	29.71 ↑	25.02 ↑	86.99 ↑	2.63 ↑
Qwen2.5-Coder-32B	**15.10**	**34.60**	**27.56**	**87.83**	2.53	**16.17** ↑	**35.48** ↑	30.22 ↑	**88.40** ↑	2.73 ↑
Codestral-22B	14.19	33.29	27.03	87.72	**2.76**	15.04 ↑	34.63 ↑	**30.32** ↑	88.02 ↑	**2.97** ↑
StarCoder	11.36	29.29	21.98	86.91	2.54	13.79 ↑	32.48 ↑	27.47 ↑	87.97 ↑	2.83 ↑

Table 3: Distribution of Generated Assertion Message Structures (%). Bold cells highlight represent the structure closest to human-written assertion messages (Ground Truth).

	Test Method Only				Test Method with Comments			
Model	**Identifier**	**String Literal**	**Combination**	**Other**	**Identifier**	**String Literal**	**Combination**	**Other**
Human (Ground Truth)	0	98.15	1.85	0	-	-	-	-
CodeLlama-13b-hf	0.46	84.72	7.88	6.94	0	90.28 ↑	5.09	4.63 ↓
Qwen2.5-Coder-32B	0	88.89	4.17	6.94	0	89.30 ↑	4.65	6.05 ↓
Codestral-22B	0.93	**92.59**	6.48	0	0	91.67 ↓	8.33 ↑	0
StarCoder	0.93	**91.20**	5.56	2.31	0.46	**95.37** ↑	4.17	0 ↓

surpass human-written references in quality. This raises concerns about the suitability of relying solely on reference-based similarity metrics. To address this, we adopt an LLM-based evaluation to rank the quality of generated messages. This method captures subjective elements such as usefulness and clarity, which are not easily measured via automatic metrics. Following the approach by Sun et al. [29], we designed a structured prompt to guide the evaluation:

Prompt updated based on developer expectations [21]

Here is a Java test method and corresponding assertion messages. Please rate each assertion message on a scale from 1 to 5, where a higher score indicates better quality. A good assertion message should:

(1) Clearly describe the reason for the assertion failure.
(2) Explicitly mention expected versus actual outcomes when relevant.
(3) Be concise, informative, and easy to understand.
(4) Help developers quickly identify the source of the problem during debugging.

Your answer should follow the format ...

The original evaluation prompt proposed by [29] was designed to assess the usefulness of generated source code comments. In our study, we modified this prompt to better suit the context of assertion message generation by aligning it with developer expectations and their understanding of what makes an assertion message clear, helpful, and high-quality [21].

Structure and Linguistics: Unlike traditional code summarization, which typically produces full natural language sentences, assertion messages exhibit more diverse structures [31]. They may consist solely of identifiers, plain string literals, or a combination of both. To capture this structural diversity, we extend our evaluation by categorizing the composition of generated messages. However, manual pattern labeling is time-consuming and does not scale well. Fortunately, recent work has shown that LLMs can achieve classification accuracy comparable to manual efforts in various software engineering tasks. For example, Tafreshipour et al. [30] used LLaMA3-70B to classify code commits messages into maintenance categories, while Pister et al. [22] used GPT-4 (gpt-4-1106-preview) to categorize developer-written prompts into multiple types. Inspired by these findings, we design a few-shot classification prompt [34] to categorize assertion messages into four structural patterns: `Identifier`, `String Literal`, `Combination`, and `Other`. The prompt includes multiple labeled *n*-examples for each category to guide the LLM's reasoning process.

Additionally, assertion messages often follow common n-gram patterns, particularly shaped by the type of assertion used. For instance, messages accompanying `assertEquals` frequently include phrases such as _"should be"_ or _"does not"_. To evaluate the linguistic alignment between LLM-generated and developer-written messages, we conduct an n-gram analysis of the outputs. Specifically, we extract and compare the top three most frequent bigrams and trigrams from both LLM-generated and reference messages.

4 Results

This section outlines the models performance using traditional text similarity metrics, semantic similarity scores, and human-aligned LLM-based evaluation. Additionally, we analyze the structural alignment of the generated assertion messages, including their composition types—Identifier, String Literal, and Combination—as well as n-gram patterns to assess consistency with developer-written messages. In Figure 2, we illustrate concrete examples (Listings 5 through 8) of assertion messages generated by each evaluated model compared to their corresponding human-written messages.

Listing 5: Example of StarCoder with comment

```
/* Verifies that the contract XML resource
   is available on the classpath. */
public void testContractResourceOnClasspath(){
    URL url = this.getClass().getClassLoader()..
    assertNotNull(url);}
```

GroundTruth: "could not find contract resource"
Generated Msg: "Contract XML resource not found on classpath"

Listing 6: Example of CodeLlama

```
public void changeAfterBeforeCommit() {
    dirty.afterModification();
    dirty.afterCommit();
    assertThat(dirty.beforeCommit(), is(true));}
```

GroundTruth: "Changes after beforeCommit should be dirty after afterCommit"
Generated Msg: dirty.beforeCommit()

Listing 7: Example of Codestral-22B

```
public void within() {
    Filter resultFilter = CompilerUtil.parseFilter
    ....
    ....
    assertTrue(resultFilter instanceof Within);}
```

GroundTruth: "Within was expected"
Generated Msg: "Expected a Within filter but got " + resultFilter.getClass().getName()"

Listing 8: Example Of Qwen2.5-Coder

```
public void testSetHead_whenFailed() {
    int maxCapacity = nextPowerOfTwo(16);
    int itemCount = 40;
    ....
    for (int i = 1; i <= itemCount; i++) {
        buffer.add(new TestSequenced(i));}
    long newSequence = 3;
    buffer.setHead(newSequence);
    assertEquals(maxCapacity, buffer.size());}
```

GroundTruth: "buffer size should not be affected"
Generated Msg: "Buffer size should not change"

Figure 2: Examples of Assertion Message Generation

4.1 RQ1: Performance

We first investigated how effectively LLMs generate assertion messages by comparing their outputs to the ground truth. Table 2 summarizes the evaluation results for each model based on various text and semantic similarity metrics (BLEU, ROUGE-L, METEOR, and BERTScore-F1) and the average human-like evaluation scores provided by an LLM evaluator (LLM-Eval).

First, all models performed below the ground truth level (LLM-Eval = 3.24), indicating room for improvement in generating meaningful assertion messages comparable to those written manually. Among the evaluated models, Codestral-22B consistently received the highest human-like evaluations, scoring 2.76 out of 5 without additional context and improving significantly to 2.97 when additional comments were provided. This suggests that Codestral's generated messages are perceived as clearer and more useful based on the developer preferences [21] compared to those generated by other models.

Second, based on the similarity metrics, Qwen2.5-Coder demonstrated the strongest overall performance. Notably, its BERTScore-F1—a metric that captures deeper semantic similarity—reached 87.83% using only the test method and increased to 88.40% when additional contextual information was provided. These results suggest that Qwen2.5-Coder generates assertion messages that are most semantically aligned with the original intent expressed by developers.

Third, CodeLlama showed the weakest performance across all evaluated metrics, both in the test-method-only and context-enhanced settings. Although the model exhibited modest improvements when additional contextual information was provided, its scores consistently lagged behind those of the other models.

Finally, all models demonstrated noticeable improvements with the addition of contextual information in the form of test comments. This positive impact is evident across all evaluation metrics and is especially pronounced in the LLM-Eval scores—for instance, Codestral improved from 2.76 to 2.97, and StarCoder from 2.54 to 2.83. These results support our ablation study findings, confirming that incorporating additional semantic context, such as developer-written comments, enables models to generate assertion messages that are more aligned, clear, and meaningful.

> **Summary of Performance:** While current LLMs show promising potential in generating informative assertion messages, they still fall short of fully matching the clarity and semantic precision of developer-written messages. Nevertheless, incorporating contextual enhancements—such as descriptive test comments—has been shown to significantly improve their performance.

4.2 RQ2: Structure and Linguistics

This section presents an analysis of the structural and linguistic patterns observed in assertion messages generated by LLMs. Specifically, we examine the types of assertion messages constructs (identifiers, string literals, combinations) compared to human-written assertion messages. Additionally, we perform an n-gram analysis to identify common linguistic patterns.

4.2.1 Assertion Message Structure Analysis. To better assess the quality and structural alignment of the assertion messages generated by LLMs, we categorized each message into one of four composition-based types (See Section 3.4):

- **Identifier**: Consists solely of code identifiers or expressions (e.g., `instances.isEmpty()`).
- **String Literal**: Purely textual, human-readable explanations (e.g., "`Set should not contain all elements`").
- **Combination**: A mixture of string literals and identifiers, often concatenated (e.g., "`Item mismatch for sequence: ` + `e.getKey()`).
- **Other**: Messages that lack meaningful structure or semantic clarity (e.g., symbols such as \\).

Table 3 summarizes the distribution of assertion message types generated by each LLM model, compared to the distribution of ground truth.

First, the ground truth messages written by developers show a strong preference for the *String Literal* structure, accounting for 98.15% of all messages. This highlights a developer tendency toward clear, descriptive, and textual assertion explanations. Only a small fraction (1.85%) of the messages used a combination of text and code identifiers. These structural patterns and developer preferences are consistent with findings from prior studies [21, 31]

Second, all evaluated LLMs predominantly generated *String Literal* assertion messages, reflecting strong alignment with human-written message structures. Codestral-22B (92.59%) and StarCoder (91.20%) achieved the highest structural alignment in the test-method-only setting, which further improved when additional contextual information (i.e., test comments) was provided—StarCoder to 95.37%. Interestingly, Codestral-22B exhibited a slightly different behavior: while test comments effectively reduced the generation of *Identifier* to zero, they also led to a minor decline in pure *String Literal* messages. Instead, the model favored the *Combination* structure, blending textual descriptions with code identifiers.

Third, while some messages fell into the *Combination* and *Other* categories, these less desirable formats were significantly reduced with the inclusion of contextual comments. For example, CodeLlama-13B-hf reduced its generation of *Other*-type messages from 6.94% to 4.63% when comments were introduced.

Finally, the near absence of *Identifier*-only messages across all models suggests that LLMs correctly infer the need for natural language explanations in assertion messages, rather than relying solely on code-based identifiers.

Summary of Message Structure: The structural analysis shows that LLMs largely mimic the compositional patterns of developer-written assertion messages, especially when aided by contextual information. These findings reinforce the value of incorporating descriptive test comments to guide models toward producing clearer and more semantically aligned assertion messages.

4.2.2 N-Gram Analysis of Assertion Messages. To gain deeper insights into the linguistic patterns and structural alignment of generated assertion messages, we conducted a detailed n-gram analysis focusing on the most frequent bigrams (two-word sequences) and trigrams (three-word sequences) produced by each LLM. Tables 4 and 5 summarize the top three most frequently occurring patterns, categorized by assertion types (`assertEquals`, `assertTrue`, and `assertFalse`) as they most occurred in the ground truth Table 1.

In Table 4 (Test Method Only), all evaluated models closely replicated the most frequent developer-written bigrams. Specifically, models consistently generated the bigram *"should be"* for `assertEquals` and `assertTrue`, and *"should not"* for `assertFalse`, closely matching human preferences.

For the most frequent trigrams, developers commonly used the phrase *"should not be"* for both `assertEquals` and `assertFalse`. Only Qwen2.5-Coder and StarCoder correctly reproduced *"should not be"* for `assertEquals`, while all models matched this trigram for `assertFalse`. Additionally, developers frequently used *"should have been"* for `assertTrue`, which was accurately reproduced only by CodeLlama. Other models introduced less meaningful alternatives, such as Qwen2.5-Coder's *"filter is not"*.

The second-most frequent bigrams revealed additional insights. Developers used *"should not"* for `assertEquals`, which was matched by Qwen2.5-Coder and StarCoder. For `assertTrue`, the developer-preferred bigram *"should have"* was correctly reproduced by CodeLlama and Codestral-22B. Interestingly, while developers used *"should be"* as the second-most frequent bigram for `assertFalse`, no model replicated it. Instead, all models produced the third-most frequent developer bigram, *"not be"*.

Table 5 (Test Method with Comments) shows the improvements achieved when additional context was provided. Models retained strong alignment for the most frequent bigrams. Notably, Qwen2.5-Coder-32B maintained its alignment, continuing to generate *"should not"* for `assertEquals`, while Codestral-22B improved from generating *"number of"* (without comments) to *"should not"* (with comments)—which matches the developer bigrams.

Certain models, particularly CodeLlama and Qwen2.5-Coder, introduced problematic or overly context-specific phrases. For instance, CodeLlama generated the unclear bigram *"point1 2"* in Table 4, while Qwen2.5-Coder produced the less meaningful trigram *"filter filter is"*, revealing difficulties in capturing semantic intent. However, the addition of contextual comments (Table 5) improved the linguistic clarity of these outputs. CodeLlama eliminated *"point1 2"* and instead produced clearer bigrams such as *"instance of"*. Similarly, Qwen2.5-Coder improved from *"filter filter is"* to *"should be empty"*.

Summary of N-gram Analysis: The analysis demonstrates that LLMs effectively replicate common human-written linguistic patterns, especially for frequent bigrams. Introducing contextual information moderately improves N-gram precision.

5 Additional Discussions and Implications

Models performance: Although LLMs are promising tools for automatically generating assertion messages, our evaluation indicates they have not yet reached the human-written messages, particularly in terms of the human baseline (LLM-Eval score of 3.24), even when augmented with test comments. Each model showed

Table 4: Top-3 Most Frequent Bigrams and Trigrams in Assertion Messages (Test Method Only)

		Bigrams			Trigrams		
Source	**Assertion Type**	1st	2nd	3rd	1st	2nd	3rd
Human	assertEquals	should be	should not	number of	should not be	should be equal	be equal to
	assertTrue	should be	should have	was expected	should have been	configuration string should	string should start
	assertFalse	should not	should be	not be	should not be	not be closed	to no be
CodeLlama-13b-hf	assertEquals	should be	but was	number of	should be the	be the same	size should be
	assertTrue	should be	should have	point1 2	should have been	an instance of	config should start
	assertFalse	should not	not be	be empty	should not be	not be empty	should be false
Codestral-22B	assertEquals	should be	number of	should not	should be equal	be equal to	should be empty
	assertTrue	should be	should have	an instance	an instance of	should be empty	should be an
	assertFalse	should not	not be	be empty	should not be	not be empty	file should not
Qwen2.5-Coder-32B	assertEquals	should be	should not	number of	should not be	size should be	should be empty
	assertTrue	should be	is not	filter is	filter is not	is not a	filter filter is
	assertFalse	should not	not be	main has	should not be	main has subscribers	not be empty
StarCoder	assertEquals	should be	should not	not be	should not be	should be equal	should not change
	assertTrue	should be	filter expected	be empty	should be empty	should be a	script didnt run
	assertFalse	should not	not be	should be	should not be	not be empty	file should not

Table 5: Top-3 Most Frequent Bigrams and Trigrams in Assertion Messages (Test Method with Comments)

		Bigrams			Trigrams		
Source	**Assertion Type**	1st	2nd	3rd	1st	2nd	3rd
Human	assertEquals	should be	should not	number of	should not be	should be equal	be equal to
	assertTrue	should be	should have	was expected	should have been	configuration string should	string should start
	assertFalse	should not	should be	not be	should not be	not be closed	to no be
CodeLlama-13b-hf	assertEquals	should be	to be	be equal	should be equal	be equal to	size should be
	assertTrue	should be	an instance	instance of	an instance of	should start with	should be empty
	assertFalse	should not	not be	be empty	should not be	not be empty	set should not
Codestral-22B	assertEquals	should be	should not	number of	should not be	should be empty	are not equal
	assertTrue	should be	be empty	did not	should start with	should be empty	be an instance
	assertFalse	should not	not be	be empty	should not be	not be empty	be empty after
Qwen2.5-Coder-32B	assertEquals	should be	should not	not be	should not be	should be empty	size should be
	assertTrue	should be	is not	filter is	filter is not	is not a	should be empty
	assertFalse	should not	not be	has subscribers	should not be	not be empty	not be closed
StarCoder	assertEquals	should be	number of	should not	incorrect number of	should be equal	be equal to
	assertTrue	should be	not found	expected a	not found in	should have been	an instance of
	assertFalse	should not	not be	be empty	should not be	not be empty	file should not

distinct strengths and weaknesses in assertion message generation. Qwen2.5-Coder excelled in semantic accuracy as indicated by high BERTScore-F1 values, suggesting superior comprehension of test intent compared to others. Codestral-22B achieved the best performance according to human-like evaluation (LLM-Eval), indicating that it produces the most developer-friendly, comprehensible messages. CodeLlama-13b-hf, conversely, consistently lagged behind in both semantic alignment and clarity, suggesting that further fine-tuning on assertion-message-specific datasets may significantly improve its performance. Therefore, selecting models aligned with specific quality objectives is essential when adopting LLMs for assertion message generation—whether the goal is maximizing semantic precision (e.g., Qwen2.5-Coder-32B) or enhancing human-readable clarity and consistency (e.g., Codestral-22B).

Contextual Information–Comments: The ablation study (in RQ1) clearly demonstrates that enriching test methods with additional semantic context significantly enhances the quality of generated assertion messages. Specifically, the inclusion of test comments notably improved human-like similarity (e.g., Codestral-22B improved from an LLM-Eval score of 2.76 to 2.97, and similar improvements were seen across other metrics and models). This reinforces the importance of well-written test documentation not only aids human developers but also directly supports more effective LLM-driven test automation practices.

Structural Alignment and Linguistic Patterns: The analysis of assertion message structures and linguistic patterns (i.e., bigrams and trigrams) reveals that LLMs are generally effective at replicating common linguistic conventions used by developers—particularly frequent phrases such as *"should be"* and *"should not be"*. While the evaluated models often adhered to these human-like patterns, structural inconsistencies were occasionally observed, including messages dominated by identifiers or code-centric artifacts (categorized as *Other*). Our results indicate that models such as CodeLlama and Qwen2.5-Coder were less reliable in avoiding such code-based

structures, especially compared to models like Codestral and StarCoder, which produced more consistently human-readable outputs. These findings underscore the need for larger and more diverse datasets of high-quality assertion messages—not only to improve benchmarking and evaluation—but also to support fine-tuning efforts aimed at reducing code-like artifacts and improving linguistic alignment with developer expectations.

Evaluation Metrics for Assertion Message: Our study utilized standard textual and semantic similarity metrics commonly adopted in code summarization tasks, including BLEU, ROUGE-L, METEOR, and BERTScore-F1. However, the nature of assertion messages differs significantly from conventional code summaries, which are typically longer and written as natural language sentences. Assertion messages, by contrast, are often shorter, more concise, and structurally diverse. As highlighted in our structural analysis (section 3.4 and section 4-RQ2), these messages may appear as short textual descriptions, code-based identifiers, or combinations of both. This inherent structural variability introduces challenges in applying standard evaluation metrics. Lexical similarity metrics such as BLEU, ROUGE-L, and METEOR rely heavily on word-level overlap, which inherently favors messages written in fluent natural language. As a result, they may undervalue assertion messages that are semantically correct but expressed using concise code-like forms or identifier-based phrasing. Although BERTScore-F1 is designed to capture semantic similarity at the embedding level, it is primarily trained on natural language text [40] and may not fully capture the details of shorter text or code-centric assertion messages. Consequently, while these metrics offer useful approximations of quality, their effectiveness may be limited when evaluating assertion messages.

6 Threats to Validity

External Validity: One concern regarding our study pertains to the generalizability of the results. Our investigation focused exclusively on Java test methods utilizing single assertion messages within JUnit4 frameworks from 20 well-engineered projects. Consequently, the observed model behaviors and insights may not directly generalize to other Java testing frameworks, such as JUnit5 or TestNG, nor to different programming languages, such as Python. Additionally, our analysis was restricted specifically to test methods containing exactly one assertion. Thus, the applicability of our findings to more complex test methods containing multiple assertions remains unexplored, representing an important direction for future research.

Internal Validity: Our evaluation of assertion message quality (LLM-Eval), structural classification, and test comments relied on GPT-4, potentially introducing evaluator bias or inaccuracies from automated classification. To mitigate this, we conducted manual reviews of LLM outputs and closely followed validated methods from prior work [22, 29, 30]. Moreover, we used default inference hyperparameters provided by each model's HuggingFace repository, without task-specific fine-tuning. Although this ensured fairness in comparing models, it may not represent each model's optimal performance. Lastly, we exclusively evaluated base-model LLMs trained with the Fill-in-the-Middle (FIM) objective, excluding instruction-tuned or commercial models (e.g., OpenAI's GPT series). While our choice established a clear baseline of the models performance–as this study represents the first systematic investigation into LLM-generated assertion messages, future research could explore hyperparameter optimization or instruction-tuned models explicitly tailored for assertion message generation, potentially uncovering further performance improvements.

7 Conclusions

In this study, we systematically evaluated four state-of-the-art Fill-in-the-Middle (FIM) large language models—Qwen2.5-Coder-32B, Codestral-22B, CodeLlama-13b-hf, and StarCoder—for their ability to generate informative assertion messages in Java test methods. Our results demonstrate that while current LLMs, notably Codestral-22B, achieve promising results, they still fall short of matching human-written assertion messages in terms of clarity and semantic accuracy. Our ablation study further highlights the significant benefit of including contextual information, such as descriptive test comments, which considerably improves the semantic precision and readability of the generated messages. Structural and linguistic analyses confirm that LLMs effectively replicate common developer patterns but reveal challenges related to semantic clarity in concise, mixed-format messages. Lastly, our findings suggest that existing textual evaluation metrics have limitations when applied to short, structurally diverse assertion messages. Our work establishes foundational insights and benchmarks to guide future research toward automated and context-aware assertion message generation, ultimately aiming to enhance test code quality and maintainability.

References

[1] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl Barr. 2024. Automatic semantic augmentation of language model prompts (for code summarization). In *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 1–13.

[2] Mistral AI. 2024. Introducing Codestral: A State-of-the-Art Code Language Model. https://mistral.ai/news/codestral-2501. Accessed: 2025-02-25.

[3] Wajdi Aljedaani, Anthony Peruma, Ahmed Aljohani, Mazen Alotaibi, Mohamed Wiem Mkaouer, Ali Ouni, Christian D Newman, Abdullatif Ghallab, and Stephanie Ludi. 2021. Test smell detection tools: A systematic mapping study. In *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*. 170–180.

[4] Ahmed Aljohani and Hyunsook Do. 2024. From fine-tuning to output: An empirical investigation of test smells in transformer-based test code generation. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 1282–1291.

[5] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*. 65–72.

[6] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. 2022. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255* (2022).

[7] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2015. Are test smells really harmful? an empirical study. *Empirical Software Engineering* 20 (2015), 1052–1094.

[8] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2024. Leveraging large language models for enhancing the understandability of generated unit tests. *arXiv preprint arXiv:2408.11710* (2024).

[9] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.

[10] Sakib Haque, Zachary Eberhart, Aakash Bansal, and Collin McMillan. 2022. Semantic similarity metrics for evaluating source code summarization. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 36–47.

[11] Yibo He, Jiaming Huang, Hao Yu, and Tao Xie. 2024. An empirical study on focal methods in deep-learning-based approaches for assertion generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1750–1771.

[12] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2. 5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[13] Vladimir Khorikov. 2020. *Unit testing principles, practices, and patterns*. Simon and Schuster.

[14] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! (2023). arXiv:2305.06161 [cs.CL]

[15] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.

[16] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210* (2023).

[17] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).

[18] Wendkûuni C Ouédraogo, Yinghua Li, Kader Kaboré, Xunzhu Tang, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F Bissyandé. 2024. Test smells in LLM-Generated Unit Tests. *arXiv preprint arXiv:2410.10628* (2024).

[19] Annibale Panichella, Sebastiano Panichella, Gordon Fraser, Anand Ashok Sawant, and Vincent J Hellendoorn. 2022. Test smells 20 years later: detectability, validity, and reliability. *Empirical Software Engineering* 27, 7 (2022), 170.

[20] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[21] Anthony Peruma, Taryn Takebayashi, Rocky Huang, Joseph Carmelo Averion, Veronica Hodapp, Christian D Newman, and Mohamed Wiem Mkaouer. 2024. On the Rationale and Use of Assertion Messages in Test Code: Insights from Software Practitioners. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 538–549.

[22] Kaiser Pister, Dhruba Jyoti Paul, Ishan Joshi, and Patrick Brophy. 2024. PromptSet: A Programmer's Prompting Dataset. In *Proceedings of the 1st International Workshop on Large Language Models for Code*. 62–69.

[23] Aniket Potdar and Emad Shihab. 2014. An exploratory study on self-admitted technical debt. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 91–100.

[24] Severin Primbs, Benedikt Fein, and Gordon Fraser. 2025. AsserT5: Test Assertion Generation Using a Fine-Tuned Code Language Model. *arXiv preprint arXiv:2502.02708* (2025).

[25] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[26] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. 2024. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 313–322.

[27] Chia-Yi Su and Collin McMillan. 2024. Distilled GPT for source code summarization. *Automated Software Engineering* 31, 1 (2024), 22.

[28] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Quanjun Zhang, et al. 2023. Automatic code summarization via chatgpt: How far are we? *arXiv preprint arXiv:2305.12865* (2023).

[29] Weisong Sun, Yun Miao, Yuekang Li, Hongyu Zhang, Chunrong Fang, Yi Liu, Gelei Deng, Yang Liu, and Zhenyu Chen. 2024. Source code summarization in the era of large language models. *arXiv preprint arXiv:2407.07959* (2024).

[30] Mahan Tafreshipour, Aaron Imani, Eric Huang, Eduardo Almeida, Thomas Zimmermann, and Iftekhar Ahmed. 2024. Prompting in the Wild: An Empirical Study of Prompt Evolution in Software Repositories. *arXiv preprint arXiv:2412.17298* (2024).

[31] Taryn Takebayashi, Anthony Peruma, Mohamed Wiem Mkaouer, and Christian D Newman. 2023. An exploratory study on the usage and readability of messages within assertion methods of test cases. In *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*. IEEE, 32–39.

[32] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).

[33] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* (2024).

[34] Yaqing Wang, Quanming Yao, James T Kwok, and Lionel M Ni. 2020. Generalizing from a few examples: A survey on few-shot learning. *ACM computing surveys (csur)* 53, 3 (2020), 1–34.

[35] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1398–1409.

[36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[37] Lucas Zamprogno, Braxton Hall, Reid Holmes, and Joanne M Atlee. 2022. Dynamic human-in-the-loop assertion generation. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2337–2351.

[38] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2020. Retrieval-based neural source code summarization. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1385–1397.

[39] Quanjun Zhang, Weifeng Sun, Chunrong Fang, Bowen Yu, Hongyan Li, Meng Yan, Jianyi Zhou, and Zhenyu Chen. 2025. Exploring automated assertion generation via large language models. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–25.

[40] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).